Tasks and Parallelism

Running work in the background and getting results back: futures, thread pools, work stealing, the C++20 coordination tools and parallel algorithms. Builds on Multithreading.

Futures and async

To **run a function in the background and get its result later**, use a `std::future<T>`.

```
#include <future>

int slowSquare(int n) { std::this_thread::sleep_for(std::chrono::seconds(1));
return n * n; }

std::future<int> f = std::async(std::launch::async, slowSquare, 7);   //
starts on another thread
doOtherWork();                                                          //
main keeps working meanwhile
int result = f.get();                                                   //
wait if needed, then get 49
```

- `f.get()` blocks until ready (once only); `wait()` and `wait_for()` do not take the value.
- **Exceptions travel**: a throwing task rethrows from `f.get()`.
- **Prefer tasks to raw threads**: `std::thread` has no result channel and terminates on exceptions.
- Pass `std::launch::async`; the default may *defer* the call to `get()`.
- An `async` future **blocks in its destructor**, so a discarded one makes the call effectively synchronous.

```cpp
// std::promise / std::future: a one-shot channel between two threads you manage yourself
std::promise<int> p;
std::future<int> f = p.get_future();
std::thread t([&p] { p.set_value(42); });    // the worker fulfils the promise (or p.set_exception( ... ))
std::cout << f.get();                        // 42
t.join();
```

`std::packaged_task` routes a function's result into a future (how pools return results).

READ MORE

Effective Modern C++ (Meyers)

- Item 35, Prefer task-based programming to thread-based (PDF pages 259 to 263)
- Item 36, Specify std::launch::async if asynchronicity is essential (PDF pages 263 to 268)
- Item 38, Be aware of varying thread handle destructor behavior (PDF pages 276 to 280)

C++ Move Semantics (Josuttis)

- Chapter 15 Move semantics in standard library types (PDF pages 229 to 250)

C++ Concurrency in Action (Williams)

- 4.2 Waiting for one-off events with futures (std::async, packaged_task, promise, shared_future) (PDF pages 104 to 115)
- 4.3 Waiting with a time limit (PDF pages 116 to 121)

`promise`, `shared_future` and `packaged_task`

TYPE	ROLE
`std::promise<T>`	the **writing** end: `set_value` or `set_exception`, once
`std::future<T>`	the **reading** end: `get()` once; move-only
`std::shared_future<T>`	a **copyable** read end: several threads can `get()` or `wait()` on the same result
`std::packaged_task<R(Args)>`	wraps a callable and routes its result (or exception) into a future: how a pool returns results

```
// promise<void>: a one-shot "go" signal for several waiting threads
std::promise<void> go;
std::shared_future<void> start = go.get_future().share();     // copyable, so
every worker gets a handle
for (int i = 0; i < 4; ++i) workers.emplace_back([start] { start.wait();
run(); });
go.set_value();                                               // all four are
released together

// packaged_task: the pool wraps a function and hands back the future
std::packaged_task<int(int)> task([](int n) { return n * n; });
std::future<int> f = task.get_future();
std::thread t(std::move(task), 7);                            // runs on any
thread
int r = f.get();                                              // 49
t.join();
```

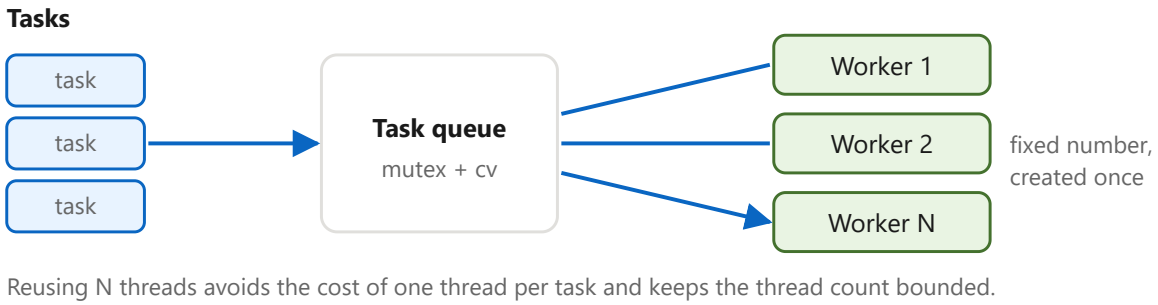
- Compiled: four workers waiting on a shared `promise<void>` had started 0 before `set_value()` and 4 after.
- A `promise` destroyed without a value gives the future a `broken_promise` exception instead of hanging forever.
- For a repeated signal use a condition variable; a promise is **one-shot**.
- A future is a *one-way, one-time* channel. If you need many messages, use a queue.

READ MORE

C++ Concurrency in Action (Williams)

- 4.2 Waiting for one-off events with futures (std::async, packaged_task, promise, shared_future) (PDF pages 104 to 115)

Thread pools and producer/consumer



Reusing N threads avoids the cost of one thread per task and keeps the thread count bounded.

Thread pool: many tasks, few long-lived workers.

A thread per task is expensive and thousands overload the scheduler. A **thread pool** starts N workers once; tasks go in a queue (**producer/consumer**).

```
class ThreadPool {
    std::vector<std::thread> workers_;
    std::queue<std::function<void()>> tasks_;
    std::mutex m_;
    std::condition_variable cv_;
    bool stop_ = false;
public:
    explicit ThreadPool(std::size_t n) {
        for (std::size_t i = 0; i < n; ++i)
            workers_.emplace_back([this] {
                for (;;) {
                    std::function<void()> task;
                    {
                        std::unique_lock<std::mutex> lk(m_);
                        cv_.wait(lk, [this] { return stop_ ||
!tasks_.empty(); });
                        if (stop_ && tasks_.empty()) return;     // finish
remaining tasks, then exit
                        task = std::move(tasks_.front());
                        tasks_.pop();
                    }                                            // lock
released BEFORE running the task
                    task();
                }
            });
    }
    void submit(std::function<void()> f) {
        { std::lock_guard<std::mutex> lk(m_); tasks_.push(std::move(f)); }
        cv_.notify_one();
    }
    ~ThreadPool() {
        { std::lock_guard<std::mutex> lk(m_); stop_ = true; }    // request
shutdown under the lock
        cv_.notify_all();                                        // wake
every sleeping worker
        for (auto& w : workers_) w.join();                       // wait for
them to finish
    }
};

ThreadPool pool(std::thread::hardware_concurrency());
pool.submit([] { std::cout << "task runs on some worker\n"; });
```

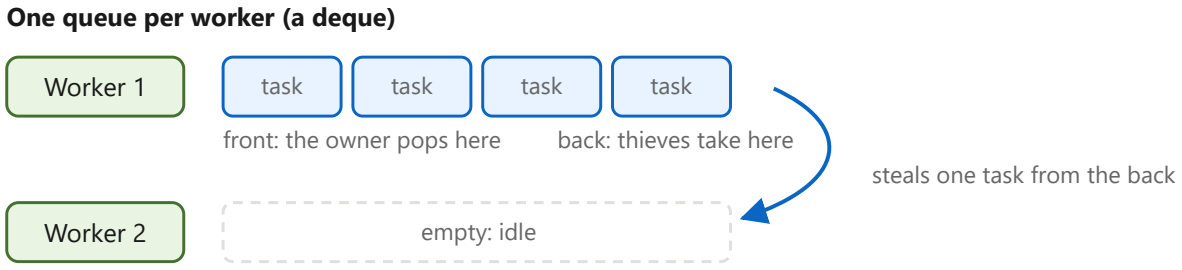
It combines everything: mutex, condition variable, stop flag, RAII shutdown. The task runs **outside** the lock, or only one could run at a time.

READ MORE

C++ Concurrency in Action (Williams)

- 9.1 Thread pools (a simple pool, waiting for tasks, work stealing in 9.1.5) (PDF pages 324 to 337)

Work stealing and sizing a pool



Owners and thieves work at opposite ends, so they rarely contend. A shared queue has one lock everyone fights over.

Work stealing: local deques keep the common case lock-light, stealing balances the load.

- **Work stealing** is how production pools (TBB, Java's fork/join) balance load: each worker pops its own deque and an idle worker **steals** from another's back.
- It suits **recursive** tasks that spawn subtasks: a spawned task goes on the spawner's own deque and stays hot in its cache.
- A simple pool with one shared queue (the previous section) is fine until that queue's lock becomes the bottleneck. Know that stealing is the next step; do not claim to have written one.

WORK	WORKERS
CPU-bound	about `hardware_concurrency()`; more is oversubscription and context switches
I/O-bound or blocking	more than the core count, because threads spend their time waiting
Mixed	separate pools, so slow blocking tasks cannot starve the fast ones

POOL DEADLOCK A task that submits a subtask to the same pool and **waits** for it can block every worker: all N workers wait for subtasks nobody is free to run. Do not block a pool thread on work that needs the same pool.

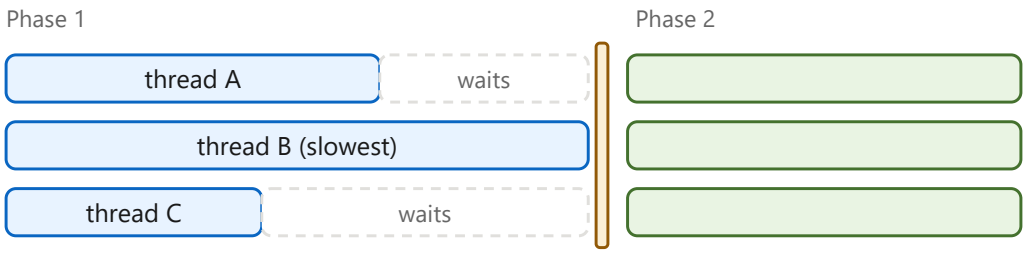
READ MORE

C++ Concurrency in Action (Williams)

- 2.4 Choosing the number of threads at runtime (PDF pages 54 to 56)
- 8.1 Techniques for dividing work between threads (PDF pages 275 to 282)
- 9.1 Thread pools (a simple pool, waiting for tasks, work stealing in 9.1.5) (PDF pages 324 to 337)

`latch`, `barrier`, `semaphore` and `stop_token` (C++20)

TOOL	IDEA	REUSABLE?
`std::latch(n)`	wait until a counter has been counted down *n* times ("N things are done")	no: one shot
`std::barrier(n)`	all *n* threads wait for each other at the end of each phase, then all go on	**yes**: phase after phase
`std::counting_semaphore<N>`	at most *k* threads in a section (a permit counter)	yes
`std::binary_semaphore`	a semaphore with one permit: signal from one thread to another	yes



The barrier (vertical bar): the last arrival releases everyone, so phase 2 starts together.
Compiled: every thread saw all three phase-1 results before it began phase 2.

A barrier separates phases of a parallel computation.

```
std::latch done(3);                                    // three workers will
count down
for (int i = 0; i < 3; ++i) workers.emplace_back([&] { work();
done.count_down(); });
done.wait();                                           // returns when the
count reaches 0

std::barrier sync(3);                                  // three threads, many
phases
auto body = [&] { phase1(); sync.arrive_and_wait(); phase2(); };

std::counting_semaphore<4> slots(2);                   // at most two threads
at once
void limited() { slots.acquire(); use(); slots.release(); }   // compiled:
peak was 2 with six threads
```

Cooperative cancellation: `jthread` and `stop_token`

```
std::jthread worker([](std::stop_token st) {
    while (!st.stop_requested()) { doWork(); }       // check the token
regularly
});
// worker.request_stop();  or just let the destructor do it: request_stop()
then join()
```

There is no safe way to *kill* a thread; you ask it to stop and it finishes its loop. `std::stop_token` is the standard channel for that request, and `std::jthread` joins in its destructor, which removes the "forgot to join" crash.

READ MORE

C++ Concurrency in Action (Williams)

- 4.4.7 to 4.4.10 Latches and barriers (the Concurrency TS versions of C++20 std::latch and std::barrier) (PDF pages 141 to 147)
- 9.2 Interrupting threads (the idea behind std::stop_token) (PDF pages 338 to 349)

Parallel algorithms (C++17)

Most standard algorithms accept an **execution policy** as a first argument. The library decides how to split the work across threads.

```
#include <execution>

std::sort(std::execution::par, v.begin(), v.end());
long long sum = std::reduce(std::execution::par, v.begin(), v.end(), 0LL);
// compiled: 500500 for 1..1000
std::transform(std::execution::par_unseq, a.begin(), a.end(), out.begin(),
f);
```

POLICY	MEANING
`seq`	sequential, as without a policy
`par`	may run on several threads
`par_unseq`	several threads and vectorised (SIMD): the function must not lock or allocate

- **Your function object must be thread-safe**: no unsynchronised writes to shared data. A data race inside a parallel algorithm is still undefined behaviour.
- `std::reduce` (not `accumulate`) is the parallel fold: it may **reassociate** the operation, so the operation must be associative (floating-point sums may differ slightly).
- It only pays for **big inputs and heavy work**; for small ranges the thread overhead dominates.
- With g++ you usually need Intel TBB installed and `-ltbb` for real parallelism; without a parallel backend the same code runs serially (as in the compiled example here).
- Exceptions escaping the callback call `std::terminate`.

CHOOSE Parallel algorithms for data parallelism over a container; a pool or `std::async` for independent tasks; a queue and a pinned thread for a latency-critical pipeline (Performance & Memory).

READ MORE

C++ Concurrency in Action (Williams)

- 10.1 to 10.3 Parallel algorithms: execution policies and the parallel standard algorithms (PDF pages 350 to 362)